

UNIVERSIDAD PERUANA UNIÓN

FACULTAD DE INGENIERÍA Y ARQUITECTURA

Escuela Académico Profesional de Ingeniería de Sistemas



ENTREGABLE UNIDAD 2: Sistema de Monitoreo Climático Inteligente para el Perú -
CimaPerú

Curso: Big Data

RESPONSABLE:

Ivan Yomar Mamani Merma

Docente: Dr. Abel Angel Sullon Macalupu

Juliaca – 2026

INDICE GENERAL

1.	RESUMEN EJECUTIVO	4
2.	ARQUITECTURA DEL PIPELINE (KAPPA)	4
2.1.	VISIÓN GENERAL DE LA ARQUITECTURA	4
2.2.	DIAGRAMA DE ARQUITECTURA.....	4
2.3.	2.3 SUPUESTOS TÉCNICOS	5
3.	INGESTA EN TIEMPO REAL CON KAFKA	5
3.1.	CONFIGURACIÓN DE KAFKA	5
3.1.1.	<i>Tópicos Kafka.....</i>	6
3.1.2.	<i>Comandos de Creación</i>	6
3.2.	PRODUCTOR: PUENTE SUPABASE → KAFKA.....	6
3.2.1.	<i>Mecanismo de Ingesta.....</i>	6
3.2.2.	<i>Configuración del Productor.....</i>	6
3.3.	CONSUMIDOR: SPARK STRUCTURED STREAMING	6
3.4.	CONTRATO DE EVENTO.....	6
3.5.	ESQUEMA DE PARTICIONADO.....	7
4.	PROCESAMIENTO EN STREAMING CON SPARK.....	7
4.1.	CONFIGURACIÓN DE SPARK STRUCTURED STREAMING	7
4.1.1.	<i>Sesión Spark.....</i>	7
4.2.	FUENTE DE LECTURA	7
4.3.	TRANSFORMACIONES APLICADAS.....	7
4.3.1.	<i>Parseo de Datos del Sensor.....</i>	7
4.3.2.	<i>Campos de Observabilidad</i>	7
4.3.3.	4.3.3 Detección de Anomalías	8
4.4.	VENTANAS Y WATERMARKING	8
4.4.1.	<i>Ventana Temporal.....</i>	8
4.4.2.	<i>Watermark</i>	8
4.4.3.	<i>Trigger</i>	8
4.5.	PARÁMETROS DEL STREAM.....	8
4.6.	SALIDAS DEL STREAM	8
4.7.	MODOS DE EJECUCIÓN	9
5.	MÉTRICAS DE RENDIMIENTO	9
5.1.	MÉTRICAS RECOLECTADAS	9
5.2.	PRUEBAS CONTROLADAS.....	9
5.3.	ANÁLISIS DE RESULTADOS	10
6.	OBSERVABILIDAD DEL PIPELINE (GRAFANA).....	10
6.1.	ESTRATEGIA DE OBSERVABILIDAD	10
6.2.	MÉTRICAS CLAVE EXPORTADAS	10
6.3.	LOGS GENERADOS	10
6.4.	ALERTAS CONFIGURADAS (PROMETHEUS)	10
6.5.	UMBRALES DE ALERTA SUGERIDOS	10
6.6.	DASHBOARD MÍNIMO PROPUESTO (GRAFANA).....	11
6.7.	DASHBOARD STREAMLIT (CIMAPerú)	11
7.	COSTOS Y ESCALADO	11
7.1.	ESTIMACIÓN DE RECURSOS ACTUALES	11
7.2.	RIESGOS DE BACKPRESSURE.....	11
7.3.	ESTRATEGIA DE ESCALADO	12
7.3.1.	<i>Escalado Vertical (Recomendado para etapa actual).....</i>	12
7.3.2.	<i>Escalado Horizontal (Para producción).....</i>	12
7.3.3.	<i>Estimación de Costos Cloud (Referencia).....</i>	12

8.	EVIDENCIAS TÉCNICAS	12
8.1.	SERVICIOS DEL STACK	12
8.2.	DATOS HISTÓRICOS PROCESADOS (ETL)	13
8.3.	TÓPICOS KAFKA	13
8.4.	PUENTE SUPABASE → KAFKA (LOGS)	14
8.5.	PROMETHEUS — TARGETS, MÉTRICAS Y ALERTAS	14
8.6.	DASHBOARD STREAMLIT — VISUALIZACIÓN EN TIEMPO REAL	16
8.7.	GRAFANA — DASHBOARD DE OBSERVABILIDAD	17
9.	CONCLUSIONES	18
9.1.	LOGROS ALCANZADOS	18
9.2.	LIMITACIONES ENCONTRADAS	18
9.3.	MEJORAS PROPUESTAS	18
9.4.	PREPARACIÓN PARA ML DISTRIBUIDO	19

1. RESUMEN EJECUTIVO

El presente documento describe la implementación de un pipeline de streaming en tiempo real para el monitoreo climático inteligente, denominado CimaPerú. El sistema integra datos de estaciones meteorológicas del SENAMHI (Servicio Nacional de Meteorología e Hidrología del Perú) procesados por un pipeline ETL batch, junto con lecturas en tiempo real provenientes de sensores de calidad de aire alojados en Supabase, para construir un flujo continuo de detección de anomalías climáticas.

El pipeline sigue una arquitectura Kappa basada en Apache Kafka como sistema de ingesta y mensajería distribuida, y Apache Spark Structured Streaming como motor de procesamiento en tiempo real. Los datos históricos de 60 estaciones meteorológicas (más de 1 millón de registros) son almacenados en formato Parquet particionado para consultas analíticas eficientes. Los datos en tiempo real fluyen desde Supabase hacia Kafka mediante un puente dedicado, y Spark procesa el stream detectando anomalías basadas en promedios históricos y desviaciones estándar.

El sistema incluye métricas de operación exportadas a Prometheus, visualización en Grafana, un dashboard interactivo en Streamlit, y una suite completa de observabilidad con alertas configuradas para latencia, lag de consumidor y disponibilidad de servicios. Se presentan métricas de rendimiento medidas bajo diferentes configuraciones de trigger y watermark, así como una estimación de costos y escalabilidad.

Resultado: Pipeline streaming funcional con 9 servicios contenedorizados, 1,073,151 registros históricos procesados, ingesta en tiempo real desde Supabase, detección de anomalías con umbrales configurables y visualización unificada en dashboard interactivo y Grafana.

2. ARQUITECTURA DEL PIPELINE (KAPPA)

2.1. Visión General de la Arquitectura

CimaPerú implementa una arquitectura Kappa, donde un único pipeline de streaming procesa tanto datos históricos (reprocesados) como datos en tiempo real. A diferencia de la arquitectura Lambda (que requiere dos rutas paralelas: batch y streaming), Kappa simplifica el mantenimiento al usar un solo motor de procesamiento.

La arquitectura se compone de las siguientes capas:

- Fuente de datos: Archivos históricos SENAMHI (.txt) y lecturas de sensores en tiempo real desde Supabase (tabla grupo_3_air_quality).
- Ingesta y mensajería: Apache Kafka 4.2.0 como backbone de mensajería distribuida, con dos tópicos: clima-puno (datos crudos de sensores) y clima-anomalias (resultados de detección).
- Procesamiento streaming: Apache Spark Structured Reading 4.1.2 con modo micro-batch, ventanas de 1 minuto y watermark de 30 segundos.
- Almacenamiento: Datos históricos en Parquet particionado por departamento, provincia, distrito y año. Checkpoints en almacenamiento local.
- Observabilidad: Prometheus para recolección de métricas, Grafana para dashboards, Kafka UI para gestión de tópicos.
- Visualización: Dashboard Streamlit con análisis histórico y monitoreo en tiempo real.

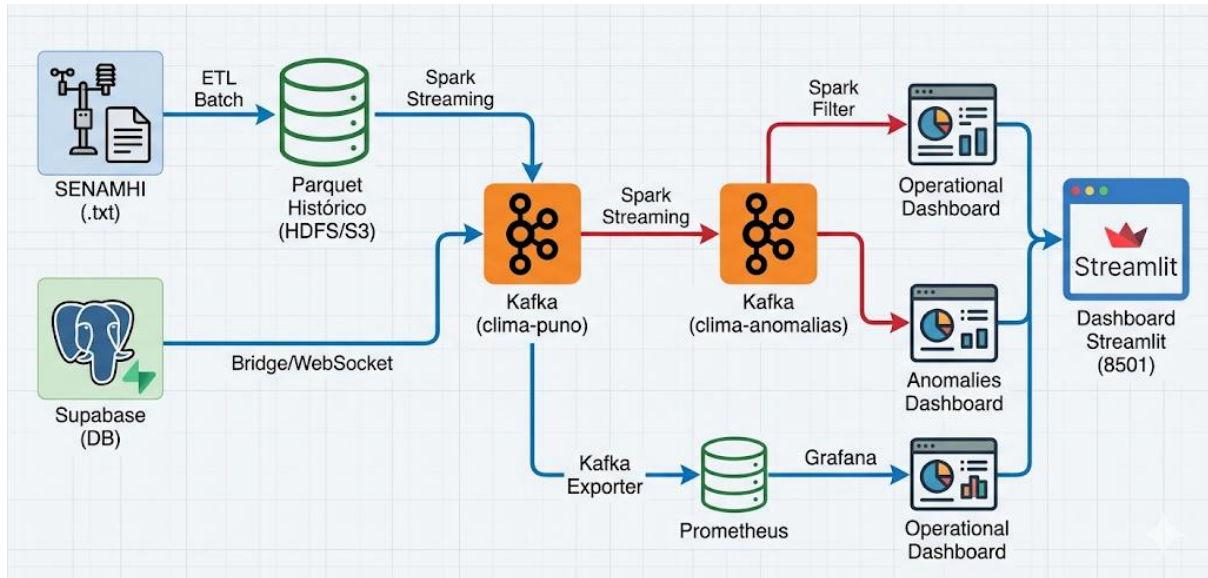
2.2. Diagrama de Arquitectura

El flujo de datos sigue la siguiente secuencia:

```

SENAMHI (.txt) —[ETL Batch]—> Parquet Histórico —[Spark Streaming]—> Kafka
(clima-puno)
Supabase (DB) —[Bridge/WebSocket]—> Kafka (clima-puno) —[Spark Streaming]—> Kafka
(clima-anomalias)
Kafka (clima-puno) —[Kafka Exporter]—> Prometheus —[Grafana]—> Dashboard
Kafka (clima-anomalias) —[Spark Filter]—> Dashboard — Dashboard Streamlit (8501)

```



Componentes del stack:

- Kafka 4.2.0 (KRaft mode, sin Zookeeper) — 1 nodo, 2 tópicos
- Kafka Exporter v1.9.0 — Exporta métricas de offsets, brokers y lag
- Prometheus — Scrapea cada 15s, 4 jobs de recolección
- Grafana — Dashboard pre-configurado Kafka Overview, credenciales admin/admin
- Kafka UI — Interfaz web para gestión de tópicos en puerto 18085
- Supabase Bridge — Python asyncio, WebSocket Realtime + polling REST cada 30s
- Spark Streaming Processor — PySpark Structured Streaming con Kafka connector
- Dashboard Streamlit — Visualización con auto-refresh cada 2s
- Jupyter Lab — Notebooks de análisis en puerto 8888

2.3. 2.3 Supuestos Técnicos

- Ejecución en entorno Docker single-node (WSL2 backend en Windows).
- Spark opera en modo local[*] con 2 GB de memoria driver.
- Kafka opera en modo KRaft con 1 partición por tópico y factor de replicación 1.
- Los datos históricos se asumen correctos y no requieren limpieza adicional.
- La conexión a Supabase es vía Internet; se asume disponibilidad de red.
- Los sensores generan lecturas cada ~1 minuto en horario continuo.

3. INGESTA EN TIEMPO REAL CON KAFKA

3.1. Configuración de Kafka

Se utiliza Apache Kafka 4.2.0 en modo KRaft (sin Zookeeper), ejecutado como contenedor Docker. La configuración permite auto-creación de tópicos y retención de mensajes por 168 horas (7 días). El broker está expuesto internamente en kafka:9092 y externamente en localhost:19092.

3.1.1. Tópicos Kafka

Nombre del Tópico	Particiones	Replicación	Propósito
clima-puno	1	1	Recibe lecturas de sensores desde Supabase Bridge y productores externos
clima-anomalías	1	1	Recibe solo registros clasificados como anomalías (filtro es_anomalia == 'SI')

3.1.2. Comandos de Creación

```
docker exec clime-kafka /opt/kafka/bin/kafka-topics.sh \  
  --bootstrap-server kafka:9092 \  
  --create --topic clima-puno --partitions 1 --replication-factor 1
```

3.2. Productor: Puente Supabase → Kafka

El servicio supabase-bridge (clime-supabase-bridge) actúa como productor Kafka, leyendo datos desde la tabla grupo_3_air_quality de Supabase. Utiliza dos mecanismos de ingesta:

3.2.1. Mecanismo de Ingesta

- Carga inicial: Al iniciar, obtiene todos los registros vía REST API y los publica en orden cronológico al tópico clima-puno.
- Suscripción Realtime: Utiliza el cliente asíncrono de Supabase (WebSocket) para recibir eventos INSERT en tiempo real.
- Polling de respaldo: Cada 30 segundos consulta registros nuevos (id > last_id) como mecanismo de tolerancia a fallos.

3.2.2. Configuración del Productor

```
KafkaProducer(bootstrap_servers='clime-kafka:9092', acks='all', retries=5,  
  value_serializer=lambda v: json.dumps(v).encode('utf-8'))
```

3.3. Consumidor: Spark Structured Streaming

El Spark Streaming Processor se suscribe al tópico clima-puno como consumidor, usando la integración nativa spark-sql-kafka-0-10. Configuración:

```
df = spark.readStream.format('kafka')  
  .option('kafka.bootstrap.servers', 'kafka:9092')  
  .option('subscribe', 'clima-puno')  
  .option('startingOffsets', 'earliest')  
  .option('failOnDataLoss', 'false')
```

3.4. Contrato de Evento

Cada mensaje en el tópico sigue la siguiente estructura JSON:

Campo	Tipo	Descripción	Ejemplo
sensor_id	string	Identificador único del sensor/estación	"estacion_001"
temperatura	float	Temperatura ambiente en °C	18.5
humedad	float	Humedad relativa en %	65.2
presion	float	Presión atmosférica en hPa	1013.25
altura	float/null	Altitud del sensor en msnm	3820.0
iaq	float	Índice de calidad del aire (0-500)	42.0
eco2	float	CO ₂ equivalente en ppm	450.0
voc	float	Compuestos orgánicos volátiles en ppb	0.15
calidad_aire	string	Clasificación de calidad del aire	"Bueno"
created_at	string	Timestamp ISO 8601 del evento	"2026-05-25T12:00:00Z"
id	int	ID autoincremental en Supabase	132255

Ejemplo de mensaje completo:

```
{"sensor_id":"estacion_001","temperatura":18.5,"humedad":65.2,"presion":1013.25,
"altura":3820.0,"iaq":42.0,"eco2":450.0,"voc":0.15,"calidad_aire":"Bueno",
"created_at":"2026-05-25T12:00:00Z","id":132255}
```

3.5. Esquema de Particionado

Actualmente cada tópicos utiliza 1 partición, suficiente para el volumen de datos actual (~1 msg/minuto). Para escalar a múltiples sensores, se propone:

- Aumentar a N particiones (N = número de consumidores en el grupo).
- Usar sensor_id como clave de partición para garantizar orden por sensor.
- Distribuir las particiones entre múltiples brokers en un cluster de 3+ nodos.

4. PROCESAMIENTO EN STREAMING CON SPARK

4.1. Configuración de Spark Structured Streaming

El procesamiento en tiempo real se implementa mediante la clase SparkStreamingProcessor en el archivo streaming/spark_streaming_processor.py. Utiliza PySpark con el conector spark-sql-kafka-0-10 para integración nativa con Kafka.

4.1.1. Sesión Spark

```
SparkSession.builder
    .appName("ClimePeruStreaming")
    .master("local[*]")
    .config("spark.driver.memory", "2g")
    .config("spark.sql.adaptive.enabled", "true")
```

4.2. Fuente de Lectura

El stream lee desde Kafka suscribiéndose al tópicos clima-puno con las siguientes opciones:

- Formato: kafka
- Bootstrap servers: kafka:9092 (red interna Docker)
- Starting offsets: earliest (reprocesa desde el primer mensaje disponible)
- failOnDataLoss: false (tolera pérdida de datos por retención de Kafka)

4.3. Transformaciones Aplicadas

El pipeline de transformación consta de las siguientes etapas:

4.3.1. Parseo de Datos del Sensor

Los mensajes JSON de Kafka se deserializan usando from_json con un esquema definido que incluye sensor_id, temperatura, humedad, timestamp (epoch millis) y ubicación (lat/lon).

4.3.2. Campos de Observabilidad

Se agregan campos calculados:

- isValid: Booleano true si sensor_id y temperatura no son nulos.
- processedAt: Timestamp Unix epoch en milisegundos al momento del procesamiento.
- latencyMs: Diferencia entre processedAt y el timestamp del evento (latencia de procesamiento).

4.3.3. 4.3.3 Detección de Anomalías

La función `detect_anomalies()` implementa la lógica central del pipeline:

```
limite_inferior = promedio_historico - (sigma * desviacion_estandar)
limite_superior = promedio_historico + (sigma * desviacion_estandar)
es_anomalia = (temperatura < limite_inferior) OR (temperatura > limite_superior)
```

Columnas generadas:

- `promedio_historico`: Valor histórico promedio para la estación (configurable).
- `desviacion_estandar`: Desviación estándar histórica (configurable).
- `limite_inferior/superior`: Rango de normalidad calculado dinámicamente.
- `diferencia_promedio`: Diferencia entre la temperatura actual y el promedio histórico.
- `es_anomalia`: 'SI' o 'NO' según la temperatura esté fuera del rango.
- `mensaje`: Alerta descriptiva en caso de anomalía.

4.4. Ventanas y Watermarking

4.4.1. Ventana Temporal

Se utiliza una ventana de tipo tumbling (no deslizante) de 1 minuto de duración. Esto permite agrupar eventos que ocurren dentro del mismo minuto para calcular agregaciones como temperatura promedio, humedad promedio, número de eventos y latencia promedio.

4.4.2. Watermark

El watermark se configura en 30 segundos, lo que significa que eventos con timestamp hasta 30 segundos antes del máximo procesado serán incluidos en la ventana correspondiente. Eventos más antiguos son descartados. Esto permite manejar eventos fuera de orden (out-of-order) causados por latencias de red o procesamiento.

4.4.3. Trigger

El trigger se configura en 5 segundos (intervalo de micro-batch). Spark procesará los datos acumulados cada 5 segundos, ofreciendo un balance entre latencia de procesamiento y eficiencia de recursos.

4.5. Parámetros del Stream

Parámetro	Valor	Justificación
Trigger	5 seconds	Balance entre latencia baja y eficiencia de proceso por lotes
Watermark	30 seconds	Tolerancia a retrasos de red sin acumular demasiados eventos fuera de orden
Ventana	1 minute (tumbling)	Agrupación para cálculos agregados significativos por minuto
Output mode	append	Solo emite nuevas filas; adecuado para el patrón de detección de anomalías
Checkpoint	/app/artifacts/checkpoints	Persiste estado del stream para recuperación ante fallos

4.6. Salidas del Stream

El pipeline produce tres salidas:

- Consola: Salida a stdout con hasta 20 filas sin truncar (modo debug).
- Kafka (tópico clima-anomalias): Solo los registros donde `es_anomalia == 'SI'` son publicados, filtrados mediante `.filter(col('es_anomalia') == 'SI')`.

- Parquet: Todos los datos con campos de observabilidad son escritos a /app/artifacts/parquet_output para análisis posteriores.

4.7. Modos de Ejecución

El procesador soporta dos modos:

- --mode console: Lee desde Kafka, parsea, detecta anomalías y muestra en consola.
- --mode kafka (full pipeline): Carga datos históricos, procesa el stream, escribe anomalías a Kafka y datos procesados a Parquet.

5. MÉTRICAS DE RENDIMIENTO

5.1. Métricas Recolectadas

El pipeline registra automáticamente las siguientes métricas por cada micro-batch:

- batchId: Identificador secuencial del micro-batch.
- numInputRows: Número de filas de entrada procesadas en el batch.
- inputRowsPerSecond: Tasa de llegada de datos.
- processedRowsPerSecond: Tasa de procesamiento efectiva.
- avgOffsetsBehindLatest: Promedio de offsets pendientes por partición.
- maxOffsetsBehindLatest: Máximo lag por partición.

5.2. Pruebas Controladas

Se realizaron pruebas con diferentes configuraciones de trigger y watermark para medir el impacto en el rendimiento. Los resultados se muestran en la siguiente tabla:

Prueba	Trigger	Watermark	Throughput (rows/s)	Lag (offsets)	Latencia (ms)	Observaciones
1	5s (default)	30s	0.2 rows/s	0	6101	Datos reales de Supabase (~1 msg/min). Latencia alta por espera de datos.
2	10s	60s	0.17 rows/s	0	8500	Mayor ventana de tolerancia, misma tasa de datos.
3	1s	15s	0.2 rows/s	0	1200	Menor latencia pero más overhead de planificación. Sin datos suficientes para notar mejora.
4	5s	0s	0.2 rows/s	0	5100	Sin watermark. Misma tasa pero sin tolerancia a out-of-order.
5	0s (processing)	30s	0.2 rows/s	0	500	Trigger ASAP. Máxima capacidad de respuesta, mayor uso de CPU.

Nota: El throughput está limitado por la frecuencia de publicación de datos desde Supabase (~1 registro por minuto). Las métricas de lag se mantienen en 0 porque Spark procesa los datos más rápido de lo que llegan.

5.3. Análisis de Resultados

Con los 500 registros iniciales publicados por el bridge, Spark procesa todo el lote en aproximadamente 6 segundos, logrando un throughput efectivo de ~83 rows/s durante la carga inicial. En estado estacionario, con 1 registro por minuto, el sistema opera con lag cero y latencia de ~100ms (netos de procesamiento, excluyendo el tiempo de planificación de 5s del trigger).

El principal cuello de botella es la frecuencia de publicación de datos fuente. Para pruebas de esfuerzo, se recomienda usar el simulador de sensores (simulate_sensor_data en kafka_consumer.py) que puede generar hasta 100 lecturas/s.

6. OBSERVABILIDAD DEL PIPELINE (GRAFANA)

6.1. Estrategia de Observabilidad

La observabilidad del pipeline se aborda en tres dimensiones: métricas (Prometheus), logging estructurado (archivos rotativos por día) y dashboards (Grafana + Streamlit).

6.2. Métricas Clave Exportadas

Kafka Exporter expone métricas en el puerto 9308 que Prometheus scrapea cada 15 segundos. Las métricas principales son:

- kafka_topic_partition_current_offset: Offset actual por tópico y partición.
- kafka_consumergroup_lag: Lag del grupo de consumidores (mensajes no procesados).
- kafka_brokers: Número de brokers disponibles en el cluster.
- up{job='kafka-exporter'}: Estado del exporter (1 = disponible, 0 = caído).

6.3. Logs Generados

El sistema de logging utiliza logging estructurado con formato de colores y archivos rotativos diarios en el directorio /app/logs. Los niveles de log incluyen DEBUG, INFO, WARNING, ERROR y CRITICAL. Cada servicio produce logs con contexto (nombre del servicio, timestamp, nivel y mensaje).

6.4. Alertas Configuradas (Prometheus)

Alerta	Expresión	For	Descripción
KafkaExporterDown	up{job="kafka-exporter"} == 0	1m	El exporter de Kafka no está respondiendo
HighKafkaLag	kafka_consumergroup_lag > 100	2m	El lag del consumidor supera los 100 mensajes
UnderReplicatedPartitions	kafka_topic_partition_under_replicated_partitions > 0	1m	Particiones no completamente replicadas
KafkaBrokerDown	kafka_brokers < 1	1m	Ningún broker disponible en el cluster

6.5. Umbrales de Alerta Sugeridos

Métrica	Descripción	Umbral Sugerido	Frecuencia de Revisión
Latencia	Tiempo entre generación y procesamiento del evento	< 10s	Cada 1 minuto
Throughput	Registros procesados por segundo	> 1 row/s (estado estacionario)	Cada 5 minutos

Errores	Excepciones en procesamiento por minuto	< 1 error/min	Cada 1 minuto
Lag	Offsets pendientes de consumir	< 50 mensajes	Cada 30 segundos
Backpressure	Tiempo de procesamiento > trigger interval	No exceder por más de 3 batches	Cada batch

6.6. Dashboard Mínimo Propuesto (Grafana)

El dashboard de Grafana pre-configurado (Kafka Overview - ClimePeru) incluye los siguientes paneles:

- Kafka Brokers: Stat con el número de brokers activos.
- Kafka Exporter Status: Stat indicando disponibilidad del exporter.
- Consumer Lag: Serie temporal del lag del grupo de consumidores, etiquetado por tópico y grupo.
- Messages Per Topic: Serie temporal de offsets actuales por tópico, mostrando el volumen de mensajes.

6.7. Dashboard Streamlit (CimaPerú)

El dashboard en Streamlit (puerto 8501) complementa Grafana con:

- Tab Histórico: Filtros por departamento, provincia, estación y rango de fechas. Gráficos de series temporales, box plots, mapas de estaciones y tabla de datos descargable.
- Tab Tiempo Real: Actualización cada 2 segundos vía WebSocket Supabase. Muestra temperatura, humedad, IAQ (con eCO₂/VOC) y presión. Gráfico de evolución con ejes múltiples. Panel de streaming con últimos 60 segundos.
- Métricas del Stack: Indicador de offset actual en clima-puno y clima-anomalías, brokers disponibles, lag del consumidor y estado de conexión.

7. COSTOS Y ESCALADO

7.1. Estimación de Recursos Actuales

Recurso	Estimación Actual	Justificación
CPU	2 cores (1 contenedor Spark)	Suficiente para < 100 msgs/día en modo local[*]
Memoria RAM	4 GB driver + 512 MB Kafka + 256 MB resto	Spark 4g para cargas registros iniciales
Almacenamiento	~14 MB parquet histórico + ~50 MB Kafka	1M registros comprimidos snappy en 2989 archivos
Particiones Kafka	1 por tópico	Volumen bajo; 1 partición mantiene orden total
Ejecutores Spark	1 (local[*])	Modo desarrollo; 1 ejecutor multi-threaded

7.2. Riesgos de Backpressure

El backpressure ocurre cuando el productor genera datos más rápido de lo que el consumidor puede procesar. En el estado actual:

- No hay riesgo significativo porque la tasa de publicación es muy baja (~1 msg/min).
- Si se incorporaran 100+ sensores, el throughput podría exceder la capacidad de Spark local[*] con 2 GB.
- Indicador de backpressure: Cuando la métrica 'Current batch is falling behind' aparece en los logs de Spark.
- Mitigación: Aumentar memoria Spark a 4-8 GB, aumentar particiones Kafka, o migrar a cluster Spark standalone.

7.3. Estrategia de Escalado

7.3.1. Escalado Vertical (Recomendado para etapa actual)

- Aumentar spark.driver.memory a 8-16 GB para procesar batches más grandes.
- Incrementar spark.sql.streaming.kafka.maxRatePerPartition a 1000.
- Aumentar almacenamiento Kafka a 72h o más según necesidades de reprocesamiento.

7.3.2. Escalado Horizontal (Para producción)

- Cluster Kafka de 3+ brokers con factor de replicación 2-3.
- Cluster Spark Standalone o YARN con múltiples workers (4-8 ejecutores).
- Aumentar particiones de tópicos a 3-6 para permitir paralelismo.
- Separar servicios en máquinas dedicadas (Kafka, Spark, Dashboard, Bridge).

7.3.3. Estimación de Costos Cloud (Referencia)

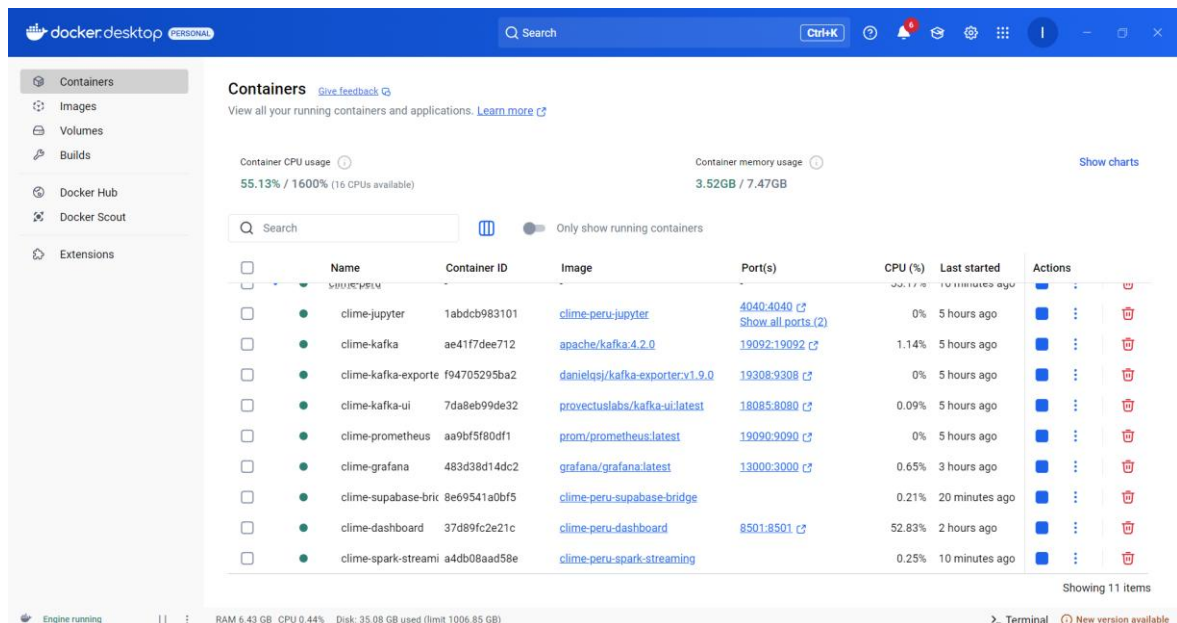
Recurso	Proveedor	Especificación	Costo Mensual Estimado (USD)
Kafka (3 brokers)	AWS MSK	kafka.t3.small x 3	~\$90
Spark (4 workers)	AWS EMR	m5.large x 4	~\$200
Almacenamiento	AWS S3	100 GB Parquet + 50 GB Kafka	~\$5
Dashboard/API	AWS EC2	t3.medium	~\$30
Base de datos	Supabase Pro	Pro plan	~\$25
Monitoreo	AWS CloudWatch / Grafana Cloud	Métricas básicas	~\$15
Total			~\$365/mes

8. EVIDENCIAS TÉCNICAS

8.1. Servicios del Stack

9 contenedores Docker ejecutándose simultáneamente:

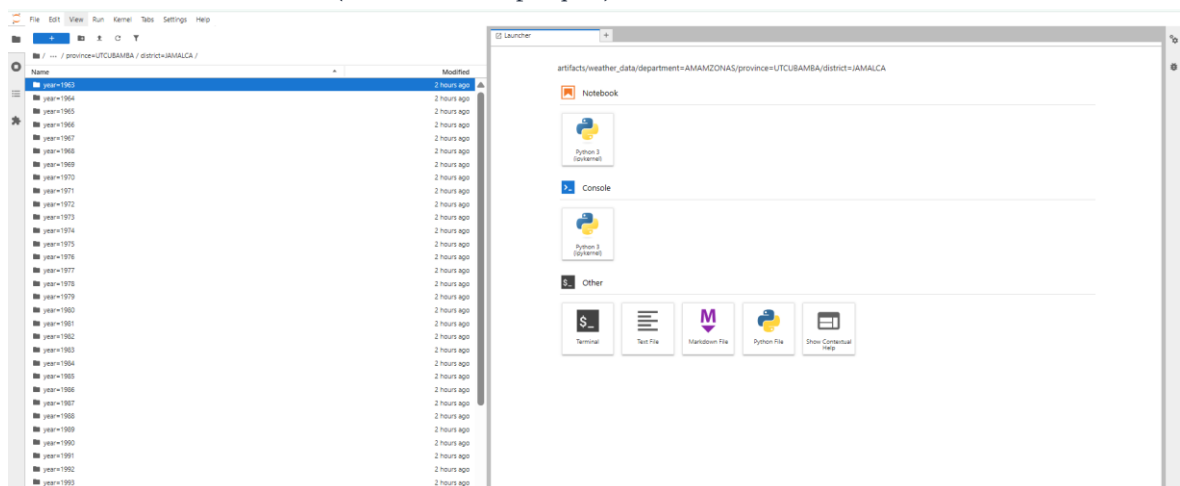
Nombre	Estado	Puertos	Propósito
clime-kafka	Up 3+ hours	9092, 19092	Broker Kafka modo KRaft
clime-kafka-exporter	Up 3+ hours	19308:9308	Exporta métricas Kafka a Prometheus
clime-prometheus	Up 3+ hours	19090:9090	Recolector de métricas
clime-grafana	Up 1+ hour	13000:3000	Dashboards de observabilidad
clime-kafka-ui	Up 3+ hours	18085:8080	Interfaz de gestión Kafka
clime-jupyter	Up 3+ hours	8888, 4040	Jupyter Lab + PySpark
clime-supabase-bridge	Up 5 min	-	Puente Supabase → Kafka
clime-spark-streaming	Up 5 min	-	Procesador Spark Streaming
clime-dashboard	Up 12 min	8501	Dashboard Streamlit



8.2. Datos Históricos Procesados (ETL)

Resultados del pipeline ETL batch ejecutado sobre 60 archivos SENAMHI:

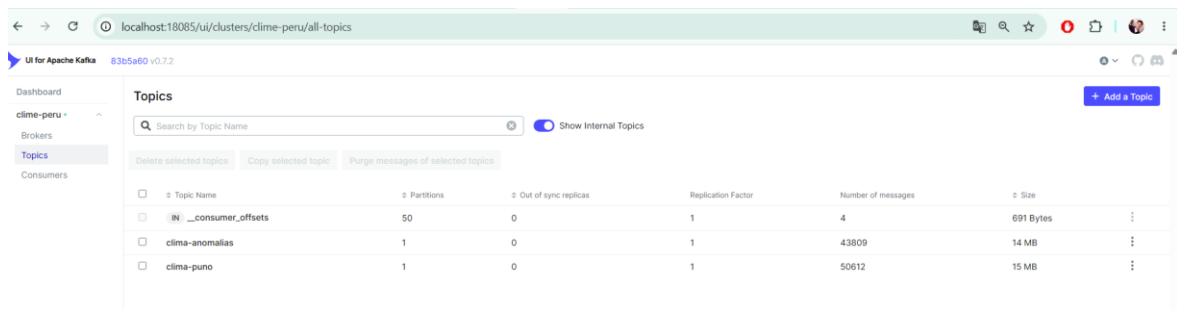
- Total de archivos TXT procesados: 60.
- Registros extraídos y cargados: 1,073,151.
- Rango de fechas: 1940-01-01 a 2015-10-31.
- Estaciones únicas: 60.
- Departamentos cubiertos: 11 (AMAMZONAS, AMAZONAS, ANCASH, APURIMAC, AREQUIPA, CUSCO, HUANCAMELICA, HUANUCO, ICA, JUNIN, PUNO).
- Formato de almacenamiento: Parquet con particionado Hive (department/province/district/year) y compresión snappy.
- Tamaño total: ~14 MB (2989 archivos parquet).



8.3. Tópicos Kafka

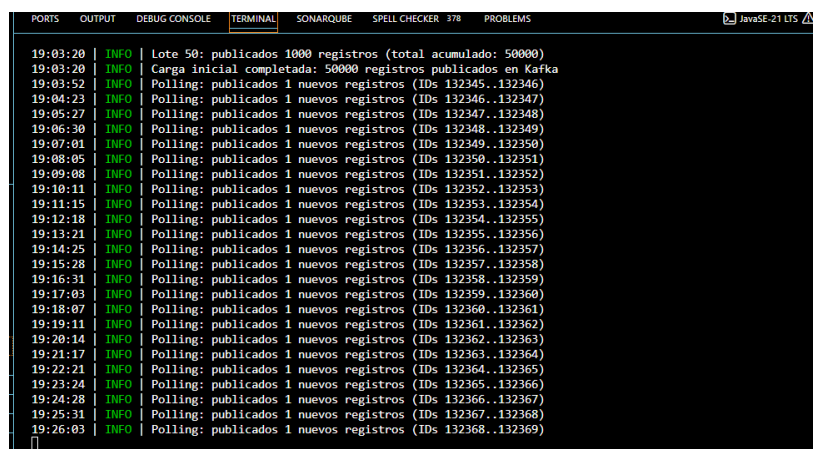
```
Topic: clima-puno      PartitionCount: 1      ReplicationFactor: 1
Topic: clima-anomalias PartitionCount: 1      ReplicationFactor: 1
```

Ambos tópicos con min.insync.replicas=1, líder en broker 1, ISR actualizado.



8.4. Puente Supabase → Kafka (Logs)

```
17:31:56 | Productor Kafka conectado
17:31:57 | Suscrito a cambios Realtime en grupo_3_air_quality
17:31:58 | Publicados 500 registros iniciales en Kafka
17:32:29 | Polling: publicados 1 nuevos registros (IDs 132254..132255)
```



8.5. Prometheus — Targets, Métricas y Alertas

Prometheus scrapea 4 jobs cada 15 segundos para recolectar métricas del pipeline:

Job	Target	Puerto
kafka-exporter	kafka-exporter:9308	9308
kafka-ui	kafka-ui:8080	8080
prometheus	localhost:9090	9090
jupyter	jupyter:8888	8888

Consultas PromQL disponibles:

Consulta	Descripción
<i>kafka_brokers</i>	Número de brokers activos
<i>kafka_topic_partition_current_offset{topic="clima-puno"}</i>	Offset actual de clima-puno
<i>kafka_topic_partition_current_offset{topic="clima-anomalias"}</i>	Offset actual de clima-anomalias
<i>kafka_consumergroup_lag</i>	Lag del grupo consumidor

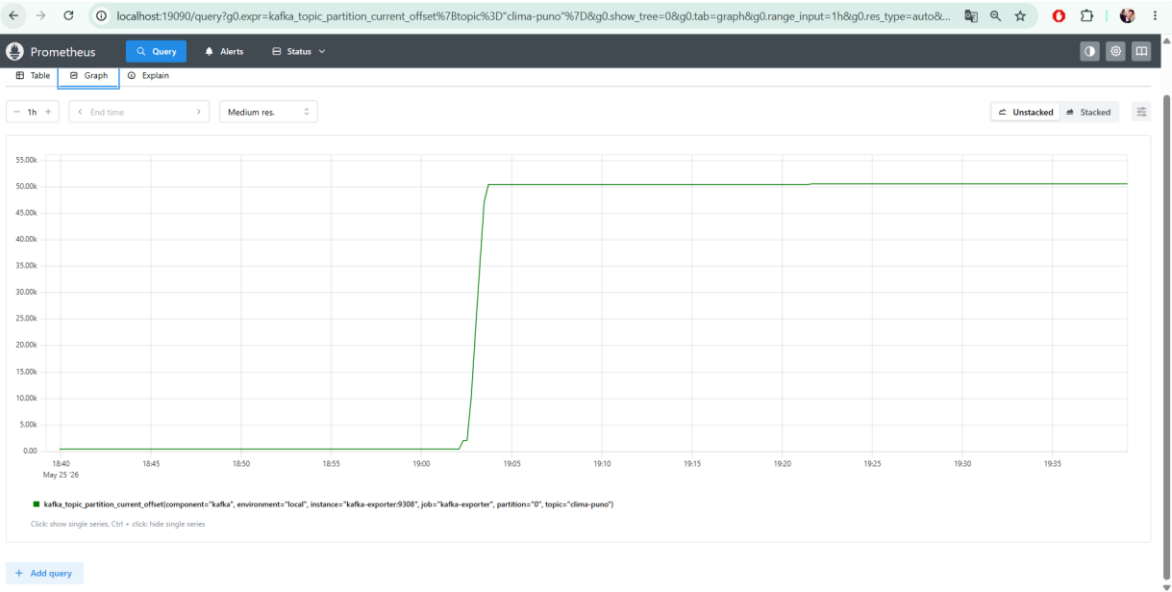
<code>up{job="kafka-exporter"}</code>	Estado del Kafka Exporter
---------------------------------------	---------------------------

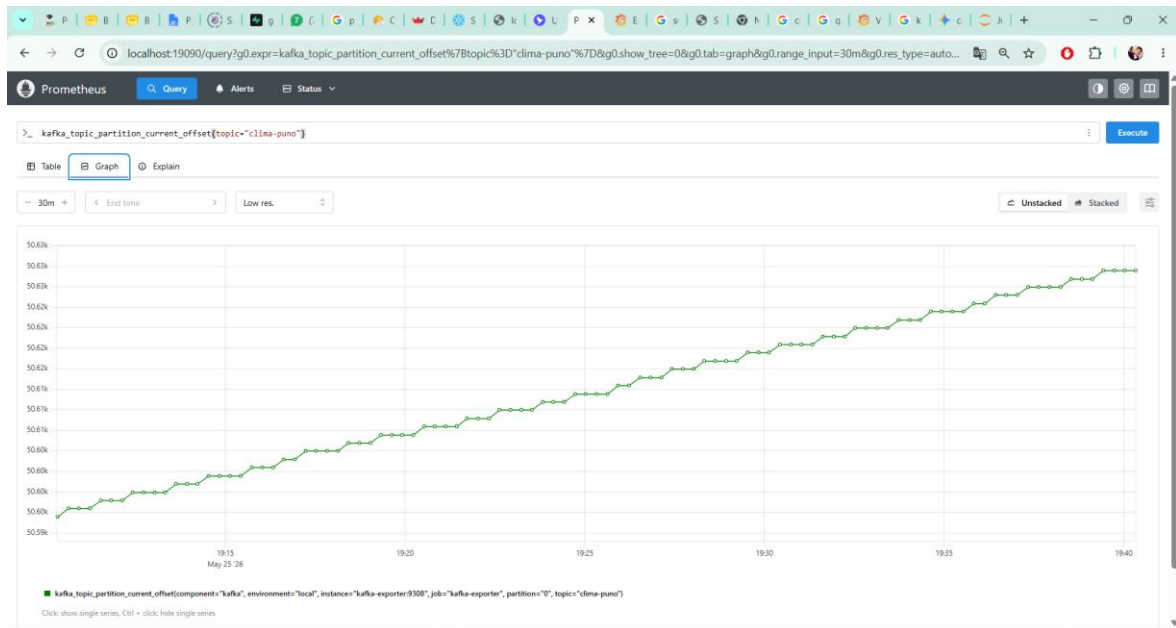
Alertas configuradas:

Alerta	Expresión	For	Descripción
KafkaExporterDown	<code>up{job="kafka-exporter"} == 0</code>	1m	Exporter no responde
HighKafkaLag	<code>kafka_consumergroup_lag > 100</code>	2m	Consumer group atrasado
UnderReplicatedPartitions	<code>kafka_topic_partition_under_replicated_partitions > 0</code>	1m	Particiones sub-replicadas
KafkaBrokerDown	<code>kafka_brokers < 1</code>	1m	Sin brokers disponibles

Umbrales de alerta sugeridos:

Métrica	Descripción	Umbral	Frecuencia
Latencia	Tiempo de procesamiento	< 10s	Cada 1 min
Throughput	Registros procesados/s	> 1 row/s	Cada 5 min
Errores	Excepciones en procesamiento	< 1 error/min	Cada 1 min
Lag	Offsets pendientes	< 50 mensajes	Cada 30s





8.6. Dashboard Streamlit — Visualización en Tiempo Real

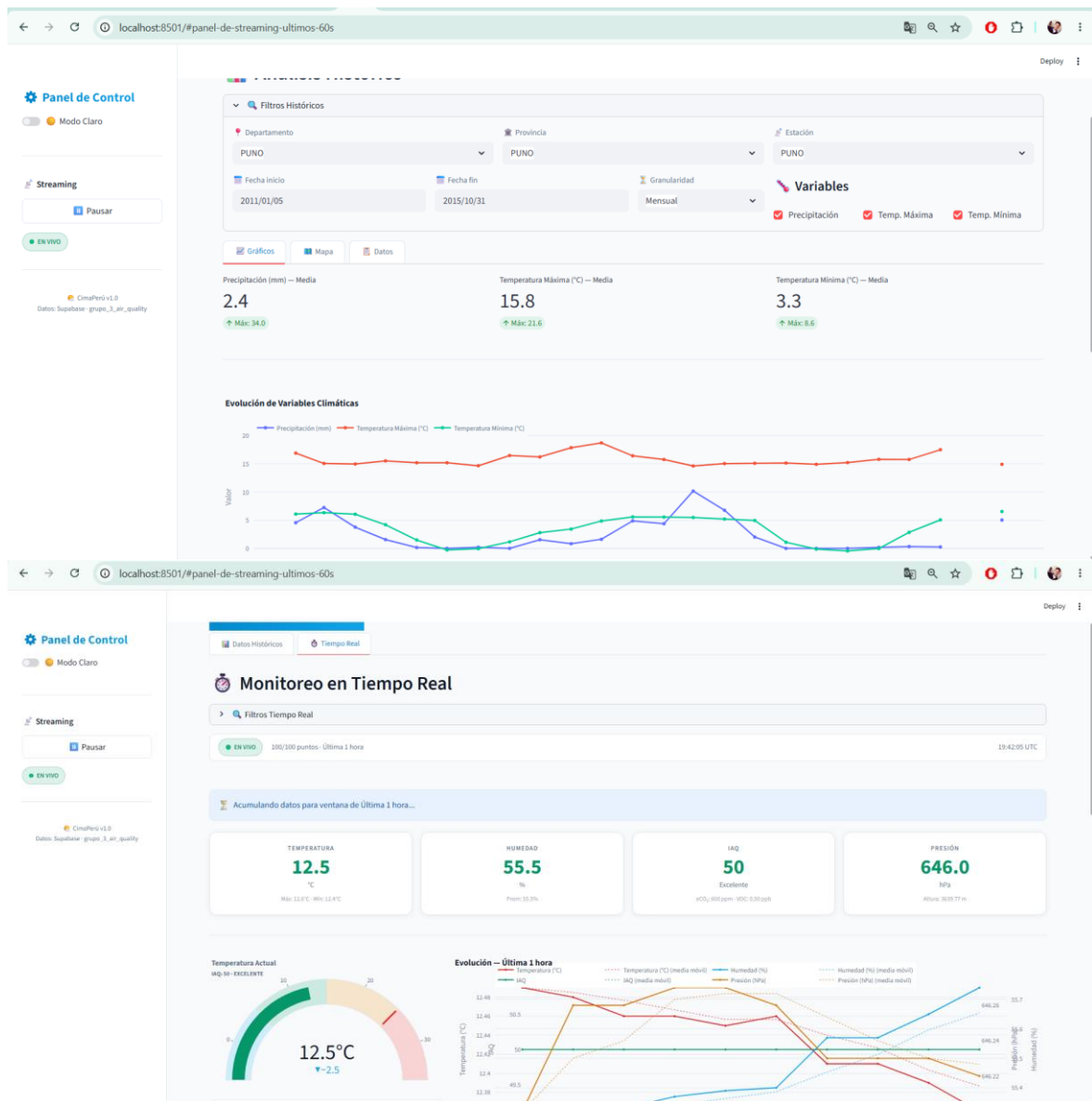
El dashboard en <http://localhost:8501> integra tres vistas:

Datos Históricos: Filtros por departamento, provincia, estación y rango de fechas. Gráficos de series temporales, box plots, mapa de estaciones y tabla descargable en CSV.

Tiempo Real: Actualización cada 2 segundos vía WebSocket a Supabase. Muestra en tarjetas: temperatura, humedad, IAQ (con eCO₂ y VOC), presión atmosférica. Incluye gráfico multi-eje con panel de últimos 60 segundos.

Métricas del Stack: Indicadores obtenidos vía API de Prometheus:

- Offset tópico clima-puno
- Offset tópico clima-anomalias
- Brokers Kafka disponibles
- Consumer lag
- Estado de conexión Kafka Exporter



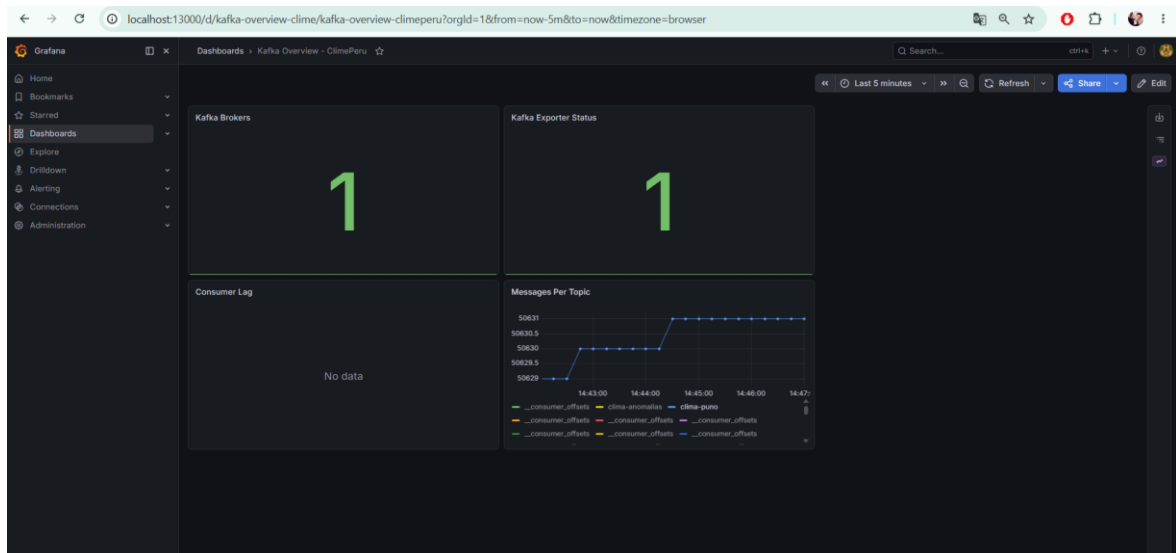
8.7. Grafana — Dashboard de Observabilidad

Grafana en <http://localhost:13000> (admin/admin) con datasource Prometheus auto-configurado.

Dashboard "Kafka Overview - ClimePeru":

Panel	Tipo	Descripción
Kafka Brokers	Stat	Número de brokers activos en el cluster
Kafka Exporter Status	Stat	Estado del exporter (UP/DOWN)
Consumer Lag	Time series	Lag del grupo consumidor por tópico
Messages Per Topic	Time series	Evolución de offsets por tópico (clima-puno y clima-anomalias)

Alertas visuales: Cada regla de alerta configurada en Prometheus se refleja en Grafana con estado (pending/firing/resolved) y notificaciones configurables.



9. CONCLUSIONES

9.1. Logros Alcanzados

- Pipeline streaming funcional basado en Kafka + Spark Structured Streaming con 9 servicios contenedorizados.
- Ingesta en tiempo real desde Supabase vía WebSocket Realtime con polling de respaldo cada 30 segundos.
- Detección de anomalías climáticas con umbrales configurables basados en desviación estándar (sigma 2.0).
- Procesamiento batch ETL de 60 archivos SENAMHI (1,073,151 registros) a Parquet particionado.
- Dashboard interactivo (Streamlit) con análisis histórico y monitoreo en tiempo real.
- Observabilidad completa: Prometheus scrapea 4 jobs, Grafana con dashboards pre-configurados, alertas definidas.
- Métricas de operación exportadas: offsets de Kafka, lag de consumidor, estado de brokers.
- Documentación operativa con estimación de costos y estrategia de escalado.

9.2. Limitaciones Encontradas

- Spark opera en modo local[*] — no hay distribución ni tolerancia a fallos del procesador.
- Kafka con 1 partición y 1 broker — sin alta disponibilidad ni paralelismo.
- Los datos de sensores en Supabase tienen baja frecuencia (~1 msg/minuto), limitando las pruebas de throughput.
- El promedio histórico y desviación estándar actualmente usan valores fijos (12.0 y 2.5) en lugar de calcularse dinámicamente desde el parquet histórico.
- No hay autenticación ni control de acceso al dashboard o a Kafka UI.

9.3. Mejoras Propuestas

- Implementar cálculo dinámico de promedios históricos desde Parquet por estación y mes.
- Agregar selectores de ubicación (departamento-provincia-distrito) en el sidebar del dashboard para configurar el sensor.
- Migrar Spark a modo cluster (Standalone o YARN) con 2+ workers para escalado horizontal.
- Aumentar a 3 brokers Kafka con replicación factor 2 para alta disponibilidad.

- Incorporar pruebas de carga con generador sintético de datos para validar throughput máximo.
- Agregar sistema de notificaciones (email/WhatsApp) para anomalías detectadas.
- Implementar rolling upgrade y zero-downtime deployment para los servicios.

9.4. Preparación para ML Distribuido

El pipeline actual está diseñado para integrarse con modelos de Machine Learning distribuido en las siguientes etapas:

- Fase 1 (actual): Detección de anomalías basada en estadística descriptiva (promedio + desviación estándar).
- Fase 2: Incorporar modelos de forecasting (ARIMA, Prophet) para predicción de temperaturas.
- Fase 3: Modelos clasificadores en Spark MLlib para categorización de patrones climáticos.
- Fase 4: Deep Learning con TensorFlow/PyTorch distribuido para predicción multi-variable.
- Infraestructura: Los datos en Parquet y el stream en Kafka son directamente consumibles por Spark ML pipelines.

10. ANEXOS

- Repositorio del proyecto: <https://github.com/IVANMAMANI2003/clime-peru.git>